Software Deployment and CI/CD with Spack

EB + EESSI + Spack Hackathon

CINECA (BO), May 19-21, 2026

Massimiliano Culpo

HPSF
HIGH PERFORMANCE SOFTWARE FOUNDATION

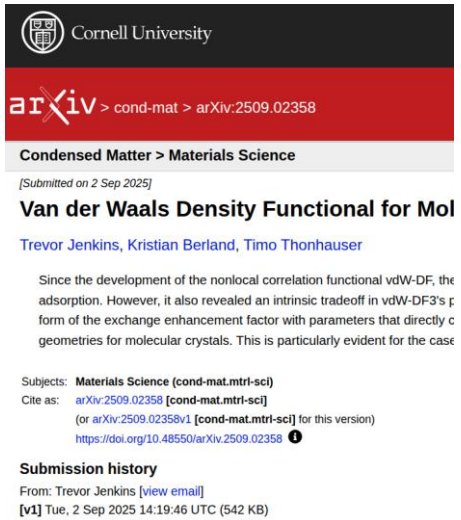
Why pipelines and CI?

Spack is extremely flexible – and that comes at a cost

- A **combinatorial** number of configurations
  - Different options
  - Different compilers, libraries, architectures
- Impossible to **always** test **all** of them
- Guarantees on **specific configurations**
  - They must be tested on each PR
  - Only check what changed

Is science always done with stable, released software?

- Often, **new algorithms** are used for research
  - They're not in any release yet
  - They are actively developed!
- Research is done in **groups**
- **Many people** need to run the **same software**
  - Artifacts should be reproducible
  - It's a waste if the same build is done repeatedly


Cornell University

arXiv > cond-mat > arXiv:2509.02358

Condensed Matter > Materials Science

[Submitted on 2 Sep 2025]

Van der Waals Density Functional for Mol

Trevor Jenkins, Kristian Berland, Timo Thonhauser

Since the development of the nonlocal correlation functional vdW-DF, the
adsorption. However, it also revealed an intrinsic tradeoff in vdW-DF3's p
form of the exchange enhancement factor with parameters that directly c
geometries for molecular crystals. This is particularly evident for the case

Subjects: Materials Science (cond-mat.mtrl-sci)
Cite as: arXiv:2509.02358 [cond-mat.mtrl-sci]
(or arXiv:2509.02358v1 [cond-mat.mtrl-sci] for this version)
https://doi.org/10.48550/arXiv.2509.02358

Submission history
From: Trevor Jenkins [view email]
[v1] Tue, 2 Sep 2025 14:19:46 UTC (542 KB)

Both problems have the same solution!

- An **environment** that defines the configuration precisely, and lives alongside the code
- A **buildcache** so that
  - unchanged dependencies are not rebuilt from sources
  - newly built binaries can be shared
- A **CI pipeline** that runs automatically when something changes

Environments

Environments are the foundation on which CI is built

spack.yaml

```yaml
spack:
  # include external configuration
  include:
  - ../special-config-directory/
  - ./config-file.yaml

  # add package specs to the `specs` list
  specs:
  - hdf5
  - libelf
  - openmpi
```

spack.lock

```
{
 "concrete_specs": {
   "6s63so2kstp3zyvjezglndmavy6l3nul": {
     "hdf5": {
         "version": "1.10.5",
         "arch": {
             "platform": "darwin",
             "platform_os": "mojave",
             "target": "x86_64"
         },
         "compiler": {
             "name": "clang",
             "version": "10.0.0-apple"
         },
         "namespace": "builtin",
         "parameters": {
             "cxx": false,
             "debug": false,
             "fortran": false,
             "hl": false,
             "mpi": true,
```

Matrices: express cross-product builds

```yaml
spack:
  definitions:
  - apps: [quantum-espresso, yambo]
  - mpis: [openmpi, mpich]
  - lapacks: [openblas, blis]

  specs:
  - matrix:
      - [$apps]
      - [$^mpis]
      - [$^lapacks]
```

```
$ spack -v concretize
==> Starting concretization pool with 8 processes
==>    28.1s [ 12%] 7imentp yambo ^openblas ^openmpi
==>    33.2s [ 25%] agxxz5t yambo ^mpich ^openblas
==>    35.4s [ 37%] q4oetvt yambo ^blis ^openmpi
==>    36.0s [ 50%] gtslocb yambo ^blis ^mpich
==>    36.6s [ 62%] llvcz7i quantum-espresso ^openblas ^openmpi
==>    37.0s [ 75%] krdgxbg quantum-espresso ^mpich ^openblas
==>    37.9s [ 87%] w4bplbr quantum-espresso ^blis ^mpich
==>    38.9s [100%] ejhjqkm quantum-espresso ^blis ^openmpi
==> Concretized 8 specs:
 -   ejhjqkm  quantum-espresso@7.5~clock~elpa+epw~fox~gipaw~ipo~l
 -   6gv5hgo      ^armpl-gcc@26.01+examples~ilp64+shared build_sy
 -   elh46z7      ^blis@2.0+blas+cblas~ilp64~scalapack build_syst
[+]  aok2qj6          ^python@3.14.3+bz2+ctypes+dbm~debug~freethr
[+]  miqstd4              ^gdbm@1.26 build_system=autotools platf
[+]  ejvhbk5              ^libffi@3.5.2 build_system=autotools pl
[+]  pst5hqw              ^pkgconf@2.5.1 build_system=autotools p
[+]  cxcmtiw              ^readline@8.3 build_system=autotools pa
[+]  p5ez6oy              ^sqlite@3.51.2+column_metadata+fts+rtre
[+]  wocoopo              ^util-linux-uuid@2.41 build_system=auto
```

Intermezzo: understanding config scopes

```
$ spack config scopes -vp
Scope           Type          Status  Path
command_line    internal      active
spack           path          active  /home/culpo/PycharmProjects/spack/etc/spack/
user            include,path  active  /home/culpo/.spack/
site            include,path  active  /home/culpo/PycharmProjects/spack/etc/spack/site/
system          include,path  active  /etc/spack/
defaults        path          active  /home/culpo/PycharmProjects/spack/etc/spack/defaults/
defaults:linux  include,path  active  /home/culpo/PycharmProjects/spack/etc/spack/defaults/linux/
defaults:base   include,path  active  /home/culpo/PycharmProjects/spack/etc/spack/defaults/base/
_builtin        internal      active
```

Spack v1.2: groups of specs in environments

```
spack:
  specs:
  - group: compiler
    specs:
    - gcc@15.2

  - group: apps
    needs: [compiler]
    specs:
    - hdf5 %gcc@15.2
    - libtree %gcc@15.2
```

```
$ spack env create stack && spack env activate stack
$ spack config edit
[ ... ]
$ spack concretize && spack install
```

Groups are named and can have dependencies

Groups are concretized independently

If a group depends on another:

- It's concretized after its dependencies
- Specs from dependencies are always *reused* in concretization

Groups can override configuration

Use of groups in public pipelines

```
- group: ml-linux-x86_64-rocm
    specs:
    - $jax_specs
    - matrix:
      - [$keras_specs]
      exclude:
      - py-keras backend=jax
      - py-keras backend=torch
    override:
      packages:
        all:
          require:
            - ~cuda
            - +rocm
            - amdgpu_target=gfx90a
```

```
109  08:23:14  ==> Concretizing the 'ml-linux-x86_64-rocm' group of specs
110  08:23:14  ==> Starting concretization pool with 8 processes
111  08:23:37  ==>   22.4s [  9%] aw7mmqf py-tensorboardx
112  08:23:39  ==>   24.3s [ 18%] 3jkv6aq py-tensorboard
113  08:23:41  ==>   27.0s [ 27%] 6zsglbl py-transformers
114  08:23:42  ==>   28.0s [ 36%] t7zf3eu py-scikit-learn
115  08:23:50  ==>   35.5s [ 45%] z2iikh6 py-jax
116  08:23:51  ==>   36.7s [ 54%] gp4h5wp py-tensorflow
117  08:23:52  ==>   37.5s [ 63%] j23kpyi py-jaxlib
118  08:23:53  ==>   39.0s [ 72%] f54twco py-keras backend=tensorflow
119  08:23:58  ==>   19.3s [ 81%] 726n2bg py-tensorflow-metadata
120  08:24:03  ==>   26.6s [ 90%] 56amp7n py-tensorflow-datasets
121  08:24:14  ==>   32.9s [100%] zcjuinb py-tensorflow-probability
122  08:24:14  ==> Concretizing the 'ml-linux-x86_64-cuda' group of specs
123  08:24:14  ==> Starting concretization pool with 8 processes
124  08:24:41  ==>   26.7s [  3%] 6zsglbl py-transformers
125  08:24:47  ==>   32.8s [  6%] j23kpyi py-jaxlib
```

Unifying the *ml-linux* pipelines:

- Reduced # of builds by ~50%
- Consolidates configuration in one place

Mirrors and Buildcaches

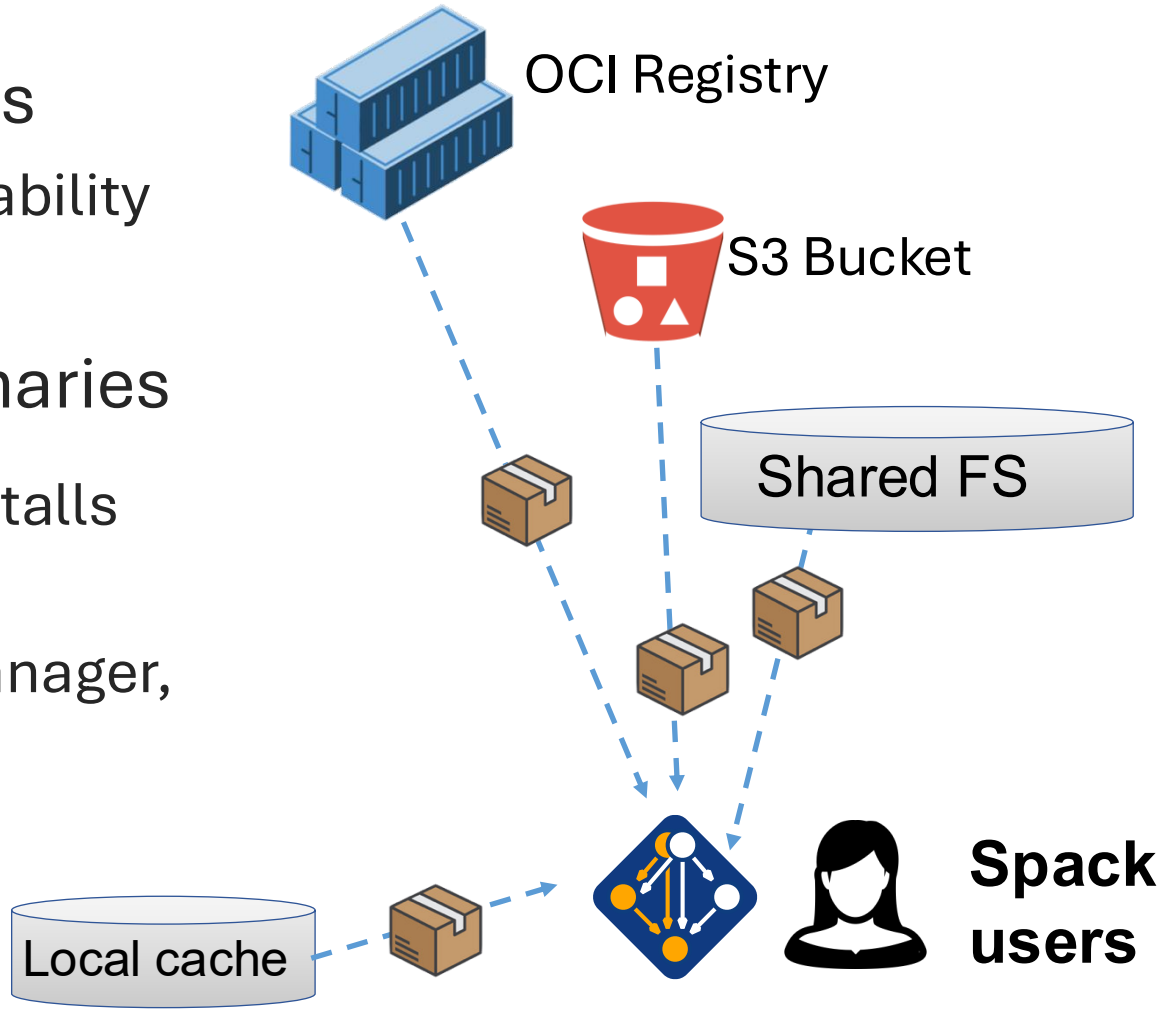
Spack is a **binary** package manager too!

A **mirror** is a copy of the package sources

- Reproduce builds without upstream availability

A **buildcache** contains pre-compiled binaries

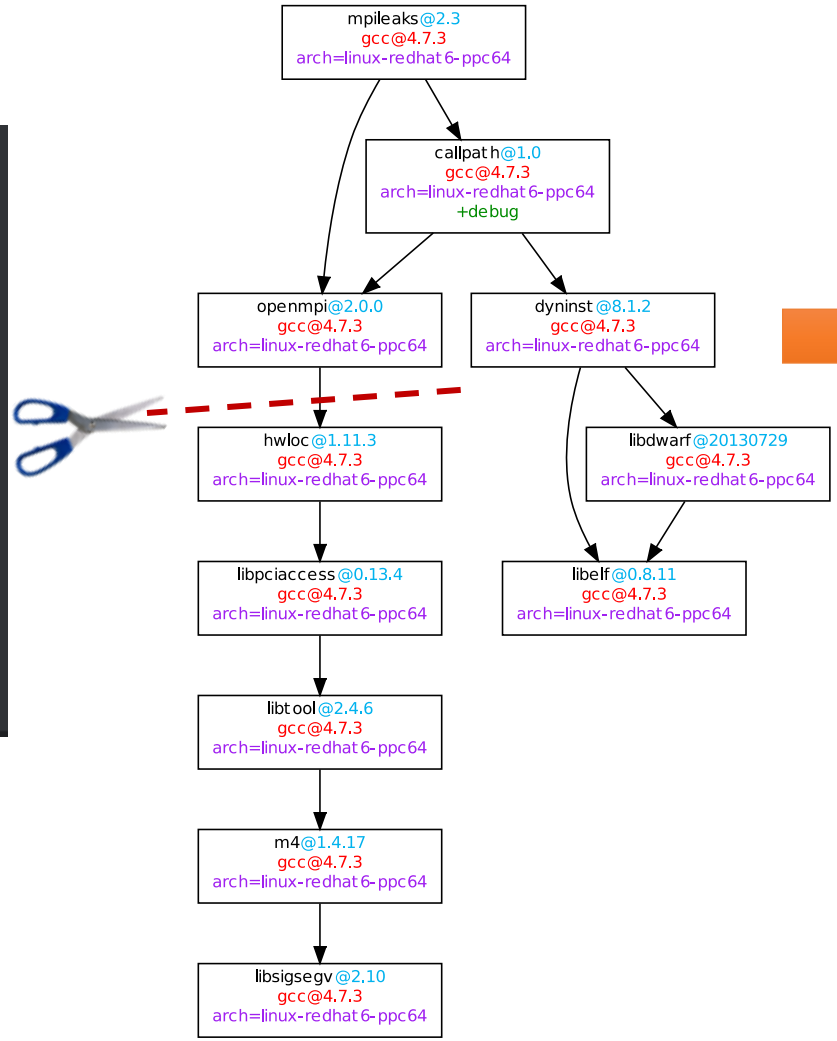
- When a matching binary is available, it installs directly – and build deps are not needed
- This turns Spack into a binary package manager, that can build from source when needed
- Packages can be GPG signed

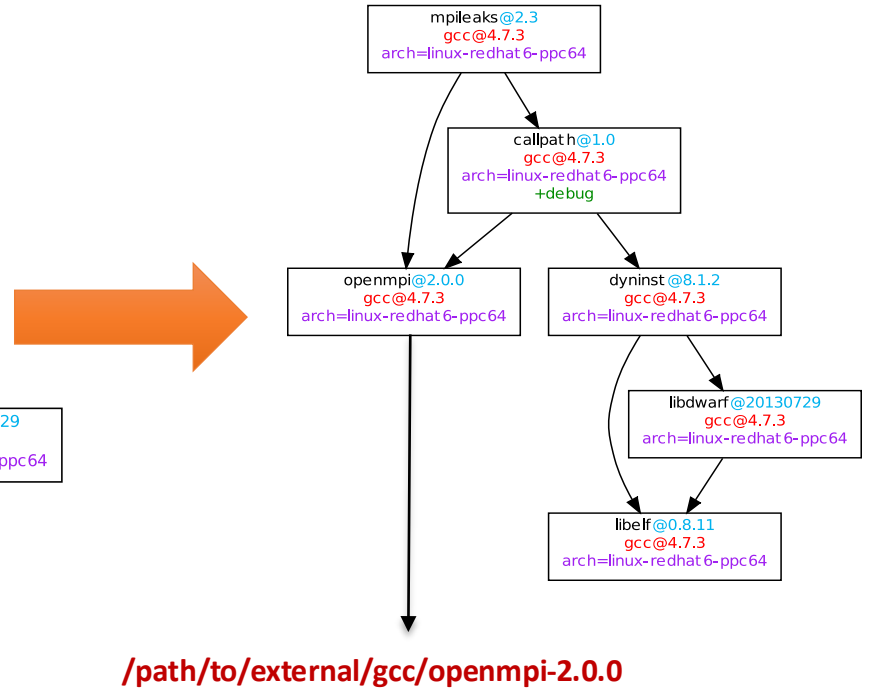


Externals

Spack v1.0 and earlier: external packages

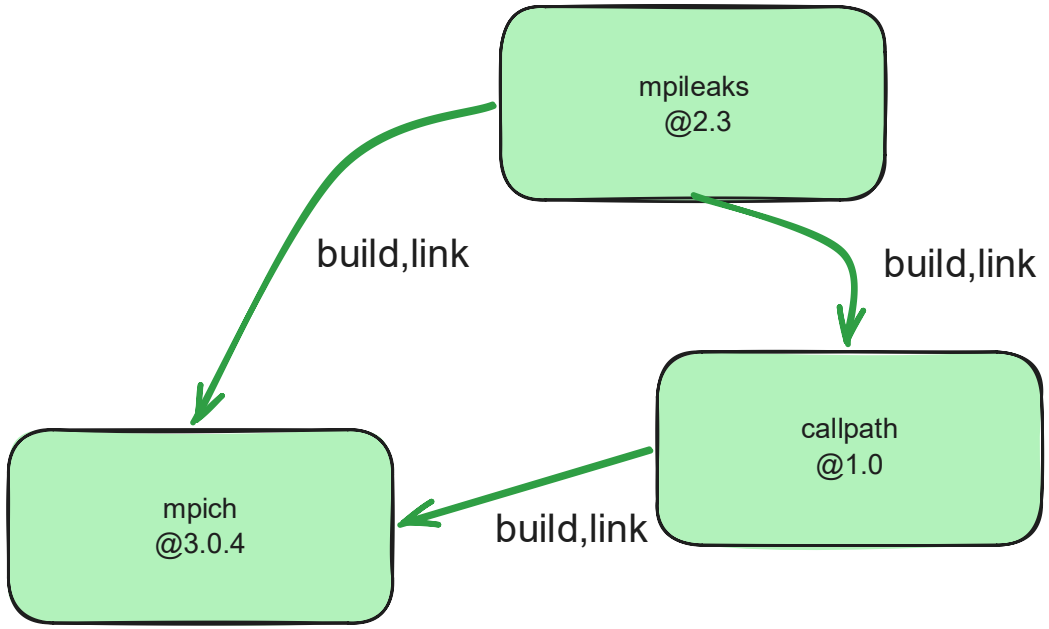
```
packages:
  openmpi:
    externals:
    - spec: openmpi@2.0.0
      prefix: /opt/openmpi-2.0
    - spec: openmpi@1.4.3
      prefix: /opt/openmpi-1.4
    buildable: false
```





Spack v1.1: externals with dependencies

```yaml
packages:
  mpileaks:
    externals:
    - spec: "mpileaks@2.3 %mpich@3 %callpath"
      prefix: /user/path
  callpath:
    externals:
    - spec: "callpath@1.0 %mpi=mpich"
      prefix: /user/path
  mpich:
    externals:
    - spec: "mpich@3.0.4"
      prefix: /user/path
```

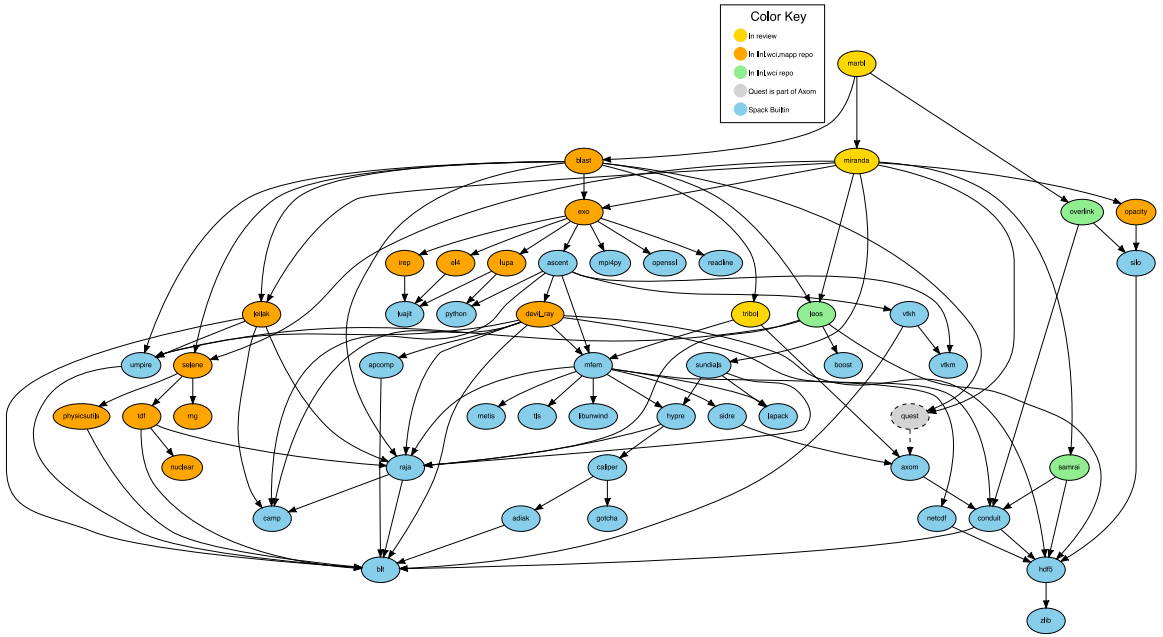


Externals can have dependencies

- inline syntax with specs (manual)
- more verbose YAML (programmatic)

Custom Repositories

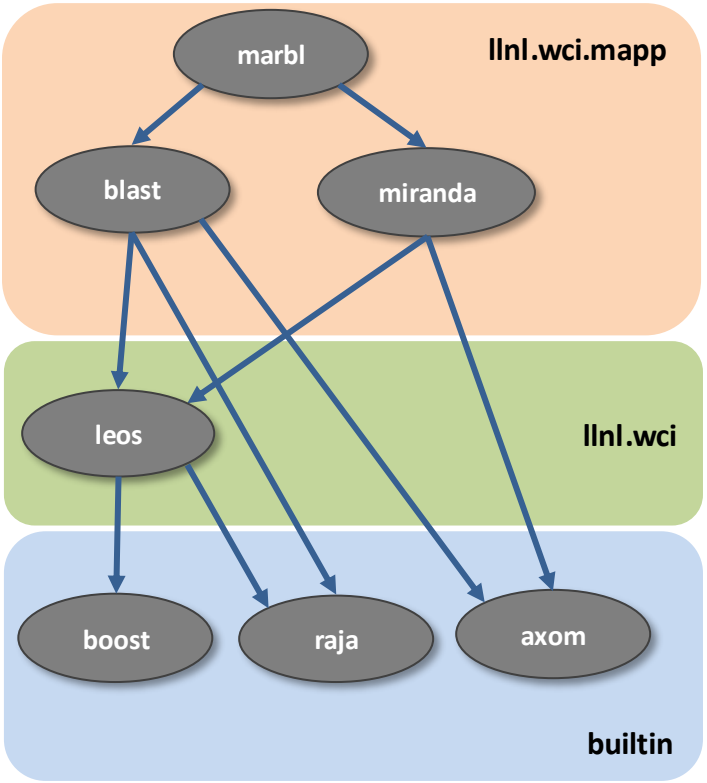
Custom repositories allow stacks to be layered



Application Packages

Common internal packages

Open-Source Spack packages




```
$ spack repo add ./my_repo
$ spack repo list
==> 2 package repositories.
my_repo    /path/to/my_repo
builtin    spack/var/spack/repos/builtin
```

Set a custom destination for a repo

```
$ spack repo list
 -  builtin        https://github.com/spack/spack-packages.git

$ spack repo set --destination /tmp/spack-pkgs builtin
==> Updated repo 'builtin'

$ spack repo update
[ ... ]
==> builtin: Updated sucessfully.

$ spack cd --repo builtin
$ pwd
/tmp/spack-pkgs/repos/spack_repo/builtin

$ spack repo list
[+] builtin    v2.2    /tmp/spack-pkgs/repos/spack_repo/builtin

$ spack config get repos
repos:
  builtin:
    destination: /tmp/spack-pkgs
    git: https://github.com/spack/spack-packages.git
```

Spack cli allows to:

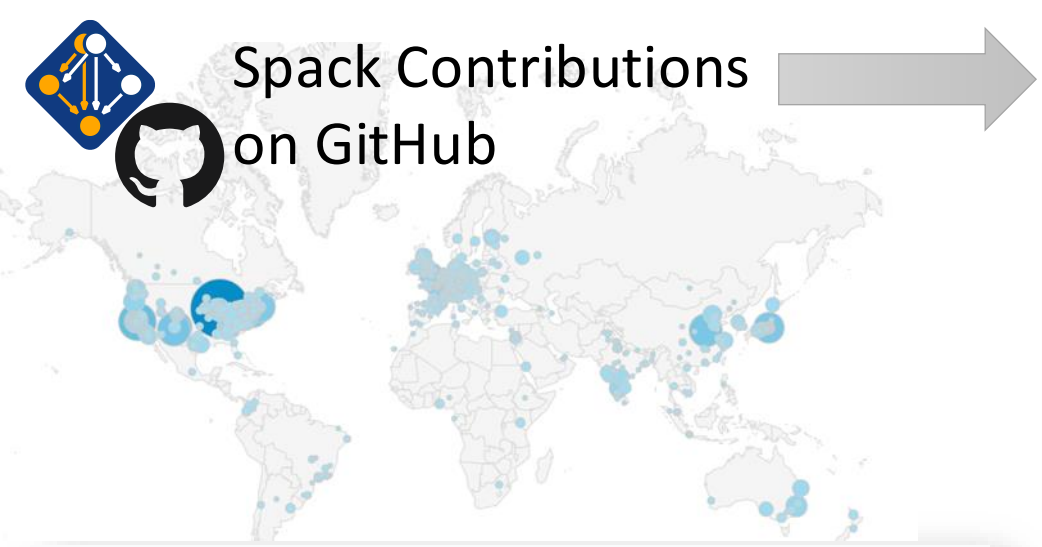
- select destination
- pin commit, tag, or branch
- update the repository

Everything is in *repos.yaml*

Puttin it all together

Spack public pipelines

Spack relies on GitLab pipelines to ensure that builds continue working



Spack Contributions on GitHub

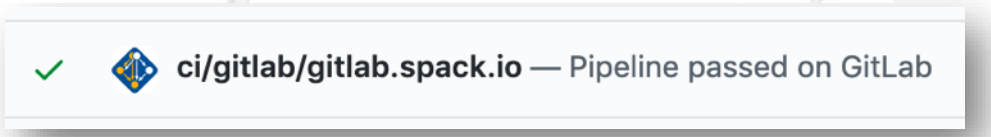
gitlab.spack.io

spack.yaml configurations

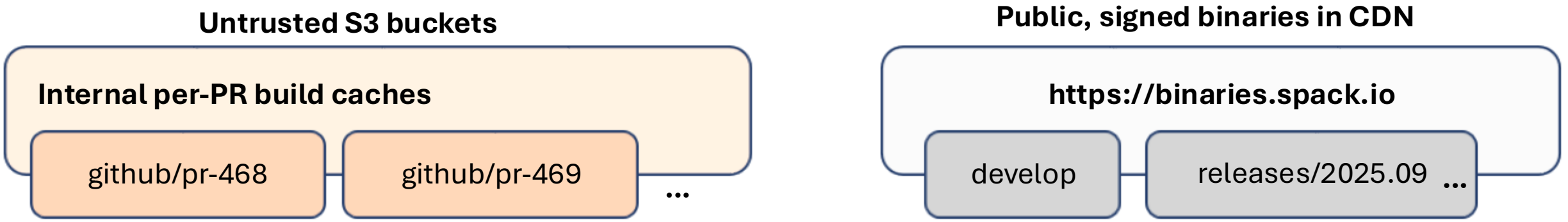
spack ci

Cray, x86_64, aarch64, MacOS pipelines with runners at **U. Oregon** or in **AWS**

GitLab CI builds (changed) packages
- On every pull request
- On every release branch



How do we sustainably manage a binary distribution?



Contributors submit package changes

- Iterate on builds in PR
- Caches prevent unnecessary rebuilds

Maintainers review PRs

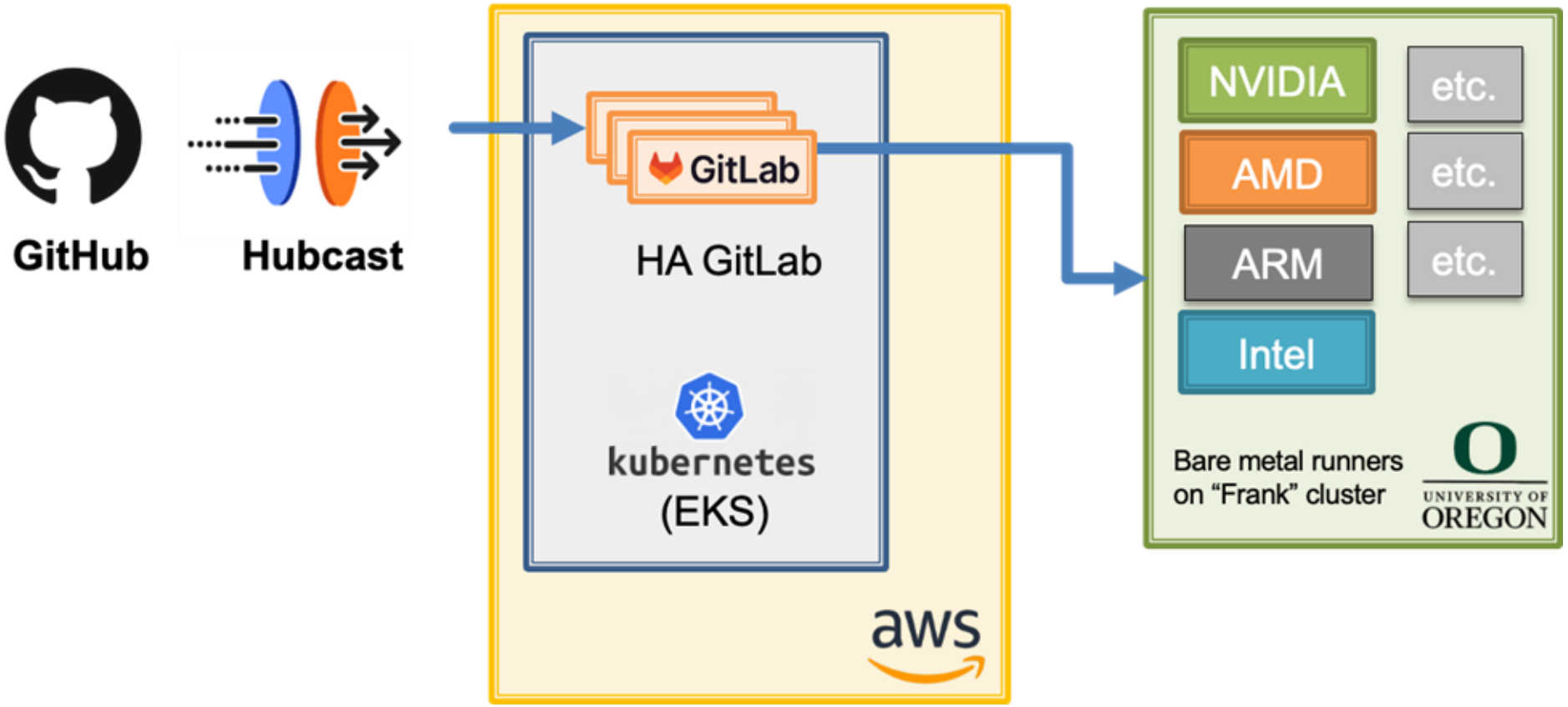
- Verify PR build succeeded
- Review package code
- Merge to develop

Rebuild and Sign

- Published binaries built ONLY from approved code
- Protected signing runners
- Ephemeral keys

- Moves bulk of binary maintenance upstream, onto PRs
  - Production binaries never reuse binaries from untrusted environment

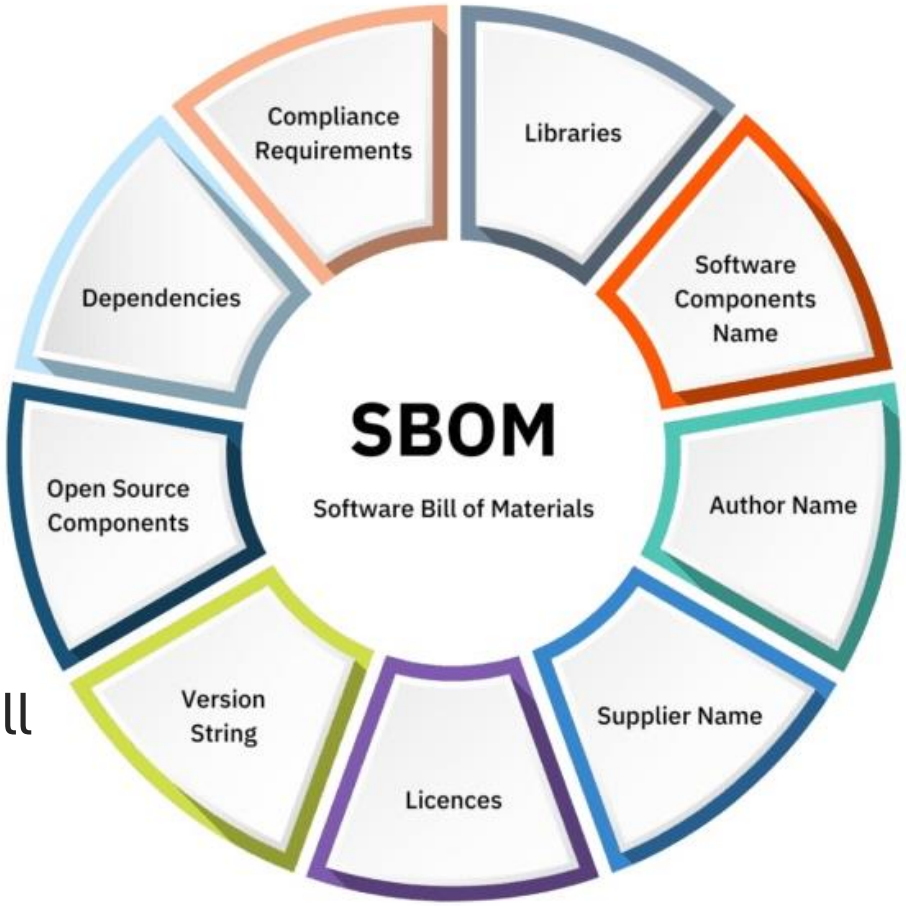
More is coming from HPSF



Thank you!

SBOMs can provide detailed provenance for packages

- Software Bill of Materials (SBOM)
  - Versions, hashes, organization of origin, etc.
  - Executive order in the last administration encouraged projects to add for security
- SBOMs enable supply chain security
  - Scanning installations for CVEs
  - Understanding outdated versions
  - Understanding where software came from
- Plan: automatically generate SBOMs with spack install
  - Expose SBOM location for scanners
  - Disclose vulnerabilities
  - Make DevSecOps easy for Spack users



Sharing a Spack instance: pip, apt and more